

Software Requirements Specification (SRS)

Personal Wealth Management Software

Project : Personal Wealth Management Software

Version : 1.0

Team : Team 9 (Section G)

Authors : Shikshith V (PES1UG24AM400), Sateesh E (PES1UG24AM397), Greeshma K (PES1UG24AM368), Rushikesh Praveen Patil (PES1UG24AM394)

Date : 07-09-2026

Status : Draft

Revision History

Version	Date	Author	Change Summary
1.0	07-09-2026	Team 9	Initial SRS draft covering all functional, non-functional, and security requirements.

Approvals

Role	Name	Signature / Email	Date
Team Lead	Shikshith V	Shikshithv@gmail.com	
Course Coordinator			

Team Role Mapping & Responsibilities

Role	Member	Core Module Focus	Key Deliverables
Frontend Lead	Shikshith V	Web UI & Client Architecture	Portfolio dashboard, net-worth charts, asset entry forms, responsive UI
Backend Lead	Sateesh E	API & Business Logic	REST APIs for asset CRUD, valuation engine, authentication & authorization
Database Lead	Greeshma K	Data Storage & Market Data	Asset & valuation schema, historical price storage, data import/export
DevOps & Security Lead	Rushikesh Praveen Patil	Integrations, Security & Deployment	Stock/gold price feed integration, alerts, security hardening, CI/CD & deployment

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features (Detailed)
- 5. Non-Functional Requirements (Detailed)
 - 5.1 Security
- 6. Quality Attributes & Acceptance Tests
- 7. System Models and Diagrams
- 8. Requirements Traceability Matrix (RTM)

1.Introduction

1.1 Purpose

This document lays out the functional, non-functional, and security requirements for the Personal Wealth Management Software. In short, the system lets a single user record, value, and keep track of a diversified personal asset portfolio — real estate, gold, stocks, and cash — from one dashboard, so they can see how their net worth changes over time instead of piecing it together across spreadsheets and bank apps.

1.2 scope

Personal Wealth Management Software is a web application built for a single user (or household) to manage a personal asset portfolio across four asset classes — real estate, gold, stocks/equity, and cash/bank balances. Specifically, the system covers:

- Asset registration, editing, and removal across all four asset classes.
- Automated valuation for market-linked assets (stocks, gold) via external price feeds, and manual/periodic valuation for illiquid assets (real estate, cash).
- A consolidated dashboard showing total net worth, asset allocation, and historical trend.
- Report generation and data export (CSV/PDF) for personal record-keeping.
- Threshold-based alerts on significant value changes.
- Secure user authentication and protection of financial data.

A few things are explicitly out of scope for this version: real money transactions, brokerage/trading execution, tax filing, multi-user/family account sharing, and integration with live banking APIs. Bank and cash balances are entered manually by the user rather than pulled from a bank feed.

1.3 Audience

This document is meant for the project team, the course instructors and QA evaluators grading this submission, and anyone who maintains or extends the system afterward.

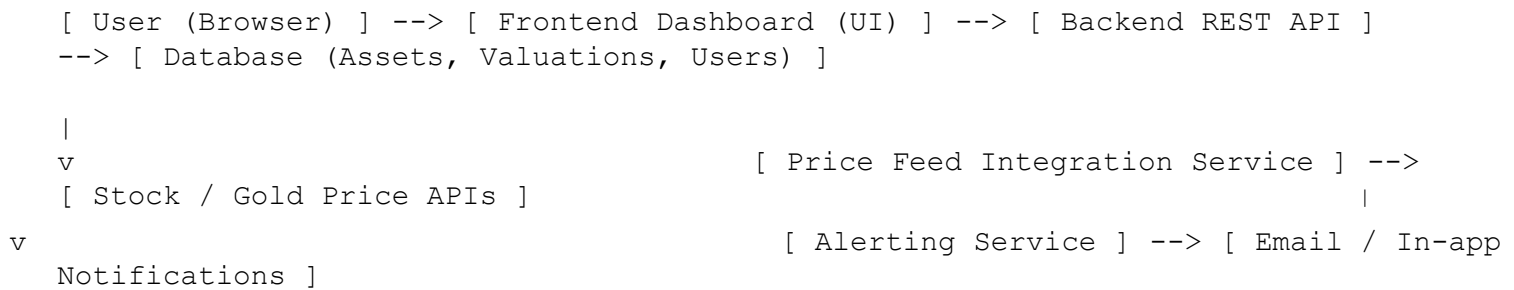
1.4 definitions

PWM (Personal Wealth Management), NAV (Net Asset Value), API (Application Programming Interface), JWT (JSON Web Token), TLS (Transport Layer Security), CRUD (Create, Read, Update, Delete), CSV (Comma-Separated Values), RTM (Requirements Traceability Matrix).

2. Overall Description

2.1 Product Perspective

The system is a standalone full-stack web application. It has four main pieces: a frontend dashboard, a backend API layer, a relational database that stores asset and valuation records, and a scheduled integration service that pulls live prices for market-linked assets like stocks and gold. A rough view of how these pieces talk to each other is shown below.



2.2 Major Product Functions (Detailed)

- User registration, login, and secure session management.
- Add / edit / delete real estate holdings (property, purchase value, current estimated value).
- Add / edit / delete gold holdings (form, weight/quantity, purchase price).
- Add / edit / delete stock holdings (ticker, quantity, average buy price).
- Add / edit / delete cash/bank balances across accounts.
- Automated valuation refresh for stocks and gold using live/periodic market price feeds.
- Manual periodic re-valuation for real estate and cash entries.
- Consolidated net-worth dashboard with asset-class allocation breakdown.
- Historical net-worth trend chart over configurable time ranges.
- Data export to CSV/PDF for personal record-keeping.
- Threshold-based alerts on significant portfolio or asset value changes.

2.3 User Roles and Characteristics

- Individual User — the main person using the app to manage their own portfolio. We're assuming basic financial literacy (they know what a stock or a gold holding is) but no technical background.
- Administrator (system maintainer) — handles price-feed configuration, keeps an eye on system health, and deals with account-level support issues.

2.4 Operating Environment

On the server side: a containerized (Docker) deployment on Linux, with the backend running on Node.js 18+ or Python 3.10+ and PostgreSQL 14+ as the database. On the client side: any modern browser (Chrome, Firefox, Safari, Edge) with a responsive layout that works on both desktop and mobile.

2.5 Constraints

- The system depends on third-party market data providers for stock and gold prices, so it's subject to their API rate limits and occasional downtime.
- Real estate valuations are manual/estimated by nature — there's no live market feed to check them against automatically.
- Single-currency (INR) support in the initial version.
- There's no integration with live banking or brokerage transaction systems; cash and real estate values are self-reported by the user.

3. External Interface Requirements

3.1 User Interfaces

The app is a responsive web dashboard. The landing page shows a summary (total net worth, allocation chart), and from there users can go to per-asset-class management screens, a historical trend view, and a settings/alerts page. Forms use inline validation and give clear error messages rather than generic failure text.

3.2 Hardware Interfaces

Nothing special — the system runs on standard server hardware and is accessed from ordinary client devices (desktop, laptop, or mobile) through a web browser. No specialized hardware is needed.

3.3 Software Interfaces

- Stock price API (e.g., NSE/BSE market data provider or a public market-data API) for live equity prices.
- Gold price API / provider for daily gold rate per gram by purity.
- Backend REST API (JSON over HTTPS) between frontend and backend.
- Email/notification service (SMTP or third-party provider) for alert delivery.

3.4 Communications

All client-server traffic and all calls to external APIs go over TLS 1.2+ (HTTPS) — no exceptions. If a price-feed call fails transiently, the integration service retries with exponential backoff; if the feed is still unavailable, it falls back to the last cached price and flags it as stale in the UI.

4. System Features (Detailed)

Each requirement below comes with acceptance criteria and a reference test case. IDs follow the PWM-F-### pattern.

4.1 User Account & Authentication

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments/ Dependencies
PWM-F-001	The system shall allow a new user to register with email and password.	Functional	High	User	AC-PWM-F-001: Valid email/password creates account. Test: TC-Auth-01	Password strength policy enforced
PWM-F-002	The system shall authenticate users via email/password and issue a session token (JWT).	Functional	High	User	AC-PWM-F-002: Valid credentials return a token; invalid credentials are rejected. Test: TC-Auth-02	Backend Lead
PWM-F-003	The system shall allow the user to reset a forgotten password via a verified email link.	Functional	Medium	User	AC-PWM-F-003: Reset link expires after 30 minutes and can be used once. Test: TC-Auth-03	Email service dependency

4.2 Asset Management (Real Estate, Gold, Stocks, Cash)

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PWM-F-010	The system shall allow the user to add a real estate asset with property name, location, purchase value, purchase date, and current estimated value.	Functional	High	User	AC-PWM-F-010: Saved asset appears in the real estate list with correct values. Test: TC-RE-01	Database Lead
PWM-F-011	The system shall allow the user to add a gold asset with form (jewellery/coin/bar), purity,	Functional	High	User	AC-PWM-F-011: Weight and purity validated; asset saved. Test: TC-Gold-01	
Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	weight (grams), and purchase price.					

PWM-F-012	The system shall allow the user to add a stock holding with ticker symbol, quantity, and average buy price.	Functional	High	User	AC-PWM-F-012: Ticker validated against known symbols; holding saved. Test: TC-Stock-01	Requires price-feed symbol lookup
PWM-F-013	The system shall allow the user to add a cash/bank balance entry with account label and current balance.	Functional	High	User	AC-PWM-F-013: Balance saved and reflected in dashboard. Test: TC-Cash-01	
PWM-F-014	The system shall allow the user to edit or delete any existing asset entry across all asset classes.	Functional	High	User	AC-PWM-F-014: Edited/deleted asset is immediately reflected in the dashboard totals. Test: TC-Asset-01	
PWM-F-015	The system shall validate asset inputs for correct data types and non-negative numeric values (e.g., weight, quantity, price ≥ 0).	Functional	High	Data Integrity	AC-PWM-F-015: Invalid input is rejected with a clear error message. Test: TC-Asset-02	

4.3 Valuation & Market Data Integration

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PWM-F-020	The system shall automatically fetch and update the current market price for each held stock at a configurable refresh interval (default: once per trading day).	Functional	High	Business	AC-PWM-F-020: Stock valuations update after each scheduled refresh. Test: TC-Val-01	DevOps Lead
PWM-F-021	The system shall automatically fetch and update the daily gold rate per gram (by purity) and revalue gold holdings accordingly.	Functional	High	Business	AC-PWM-F-021: Gold valuation reflects the latest fetched rate. Test: TC-Val-02	
PWM-F-022	The system shall allow the user to manually update the estimated value of real estate assets at any time.	Functional	Medium	User	AC-PWM-F-022: Manual update timestamp recorded. Test: TC-Val-03	
PWM-F-023	The system shall retain a historical time-series of each asset's valuation for trend analysis.	Functional	High	Database Lead	AC-PWM-F-023: Valuation history queryable by date range. Test: TC-Val-04	
PWM-F-024	If a price-feed API call fails, the system shall use the last successfully cached price and flag it as stale in the UI.	Functional	Medium	Reliability	AC-PWM-F-024: Stale indicator shown when feed is unavailable. Test: TC-Val-05	

4.4 Dashboard & Reporting

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PWM-F-030	The system shall display total net worth as the sum of current valuations across all asset classes.	Functional	High	User	AC-PWM-F-030: Total matches sum of individual asset valuations. Test: TC-Dash-01	Frontend Lead
PWM-F-031	The system shall display an asset-allocation breakdown (percentage by asset class) as a chart.	Functional	High	User	AC-PWM-F-031: Chart percentages sum to 100%. Test: TC-Dash-02	
PWM-F-032	The system shall display a historical net-worth trend chart over selectable ranges (1M, 6M, 1Y, All).	Functional	Medium	User	AC-PWM-F-032: Chart updates correctly for each selected range. Test: TC-Dash-03	
PWM-F-033	The system shall allow the user to export their full asset list and valuation history as CSV and PDF.	Functional	Medium	User	AC-PWM-F-033: Exported file contains all current asset records. Test: TC-Export-01	

4.5 Alerts & Notifications

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PWM-F-040	The system shall allow the user to configure a threshold percentage change in total net worth or a single asset's value that triggers an alert.	Functional	Medium	User	AC-PWM-F-040: Threshold saved and applied on next valuation cycle. Test: TC-Alert-01	DevOps Lead
PWM-F-041	The system shall send an email (or in-app) notification when a configured threshold is breached.	Functional	Medium	User	AC-PWM-F-041: Notification delivered within 5 minutes of breach detection. Test: TC-Alert-02	
PWM-F-042	The system shall throttle repeated alerts for the same asset to at most once per 24 hours.	Functional	Low	Operations	AC-PWM-F-042: No duplicate alert for the same trigger within 24h. Test: TC-Alert-03	

5. Non-Functional Requirements (Detailed)

The NFRs below are all measurable and each one is tied to a concrete test plan. IDs follow PWM-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
PWM-NF-001	Dashboard page shall load and render total net worth within 2 seconds under normal load.	Performance	High	90th percentile page load $\leq$ 2s in production-like test. Test: TC-Perf-01
PWM-NF-002	Stock and gold valuation refresh jobs shall complete for up to 500 tracked	Performance	Medium	Batch job duration measured in staging. Test: TC-Perf-02
Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
	assets within 5 minutes.			
PWM-NF-003	The system shall maintain 99.5% monthly uptime, excluding scheduled maintenance windows.	Reliability	High	Uptime reports show $\geq$ 99.5% per month. Test: Ops reports
PWM-NF-004	The system shall remain usable on screen widths from 360px (mobile) to desktop resolutions.	Usability	Medium	Manual responsive test across breakpoints. Test: TC-UX-01
PWM-NF-005	The system shall be able to scale to support at least 1,000 registered users without redesign of the data model.	Scalability	Medium	Load test with 1,000 simulated user portfolios. Test: TC-Scale-01

5.1 Security

5.1.1 Security Objectives

- Keep the user's financial data — asset values, holdings — confidential, both in transit and at rest.
- Make sure only an authenticated, authorized user can view or modify their own portfolio data — nobody else's.
- Protect asset and valuation records from unauthorized tampering.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
PRJ-SR-001	TLS 1.2+ shall be mandatory for all client-server and server-to-external-API network connections.	Security	High	AC-SR-001: All endpoints reject plain HTTP. Test: TC-Sec-01
PRJ-SR-002	User passwords shall be stored using a salted, one-way hash (e.g., bcrypt/argon2) and never in plaintext.	Security	High	AC-SR-002: Password field in DB is a hash, not plaintext. Test: TC-Sec-02
PRJ-SR-003	Session tokens (JWT) shall expire after a configurable period (default 24h) and be invalidated on logout.	Security	High	AC-SR-003: Expired/invalidated token is rejected by the API. Test: TC-Sec-03
PRJ-SR-004	The system shall enforce authorization checks so a user can only access asset records they own.	Security	High	AC-SR-004: Cross-user data access attempt returns 403. Test: TC-Sec-04
PRJ-SR-005	The system shall lock an account for 15 minutes after 5 consecutive failed login attempts and log the event.	Security	Medium	AC-SR-005: 6th consecutive failed attempt is blocked and logged. Test: TC-Sec-05

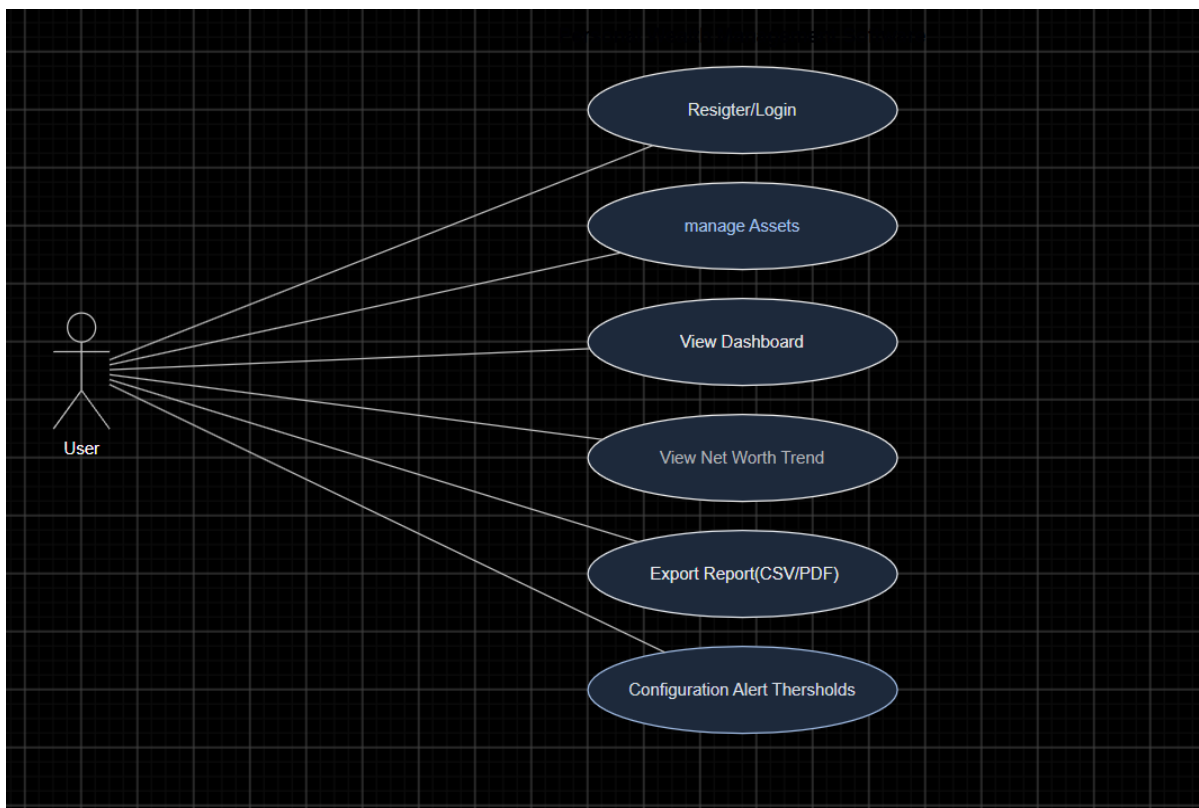
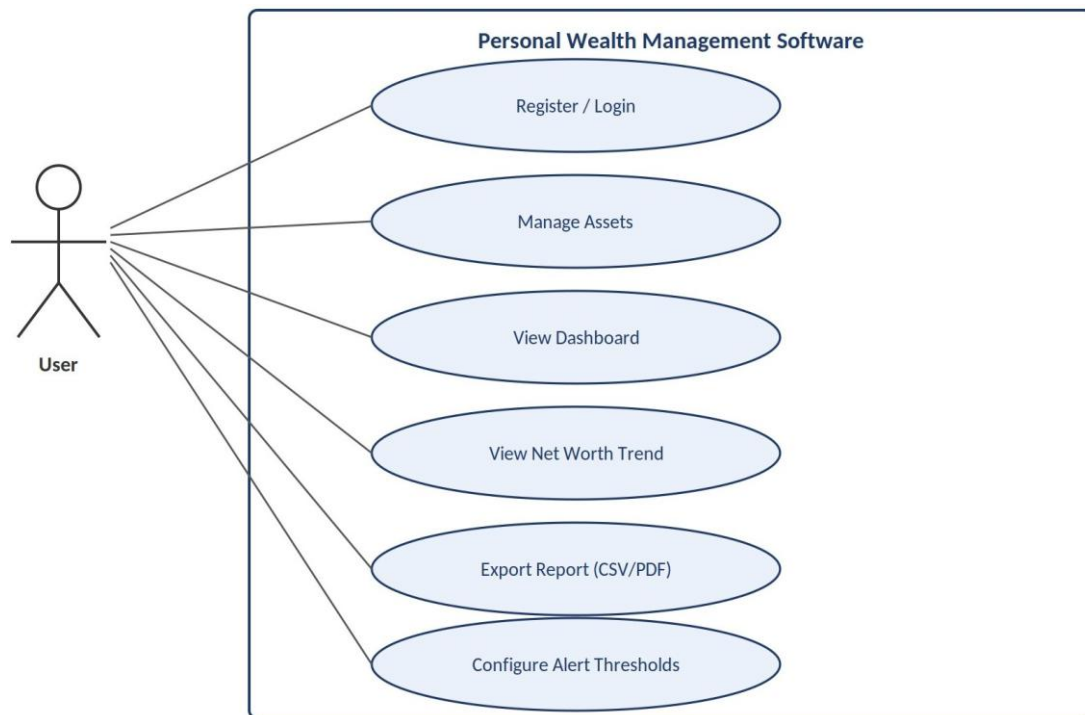
6. Quality Attributes & Acceptance Tests

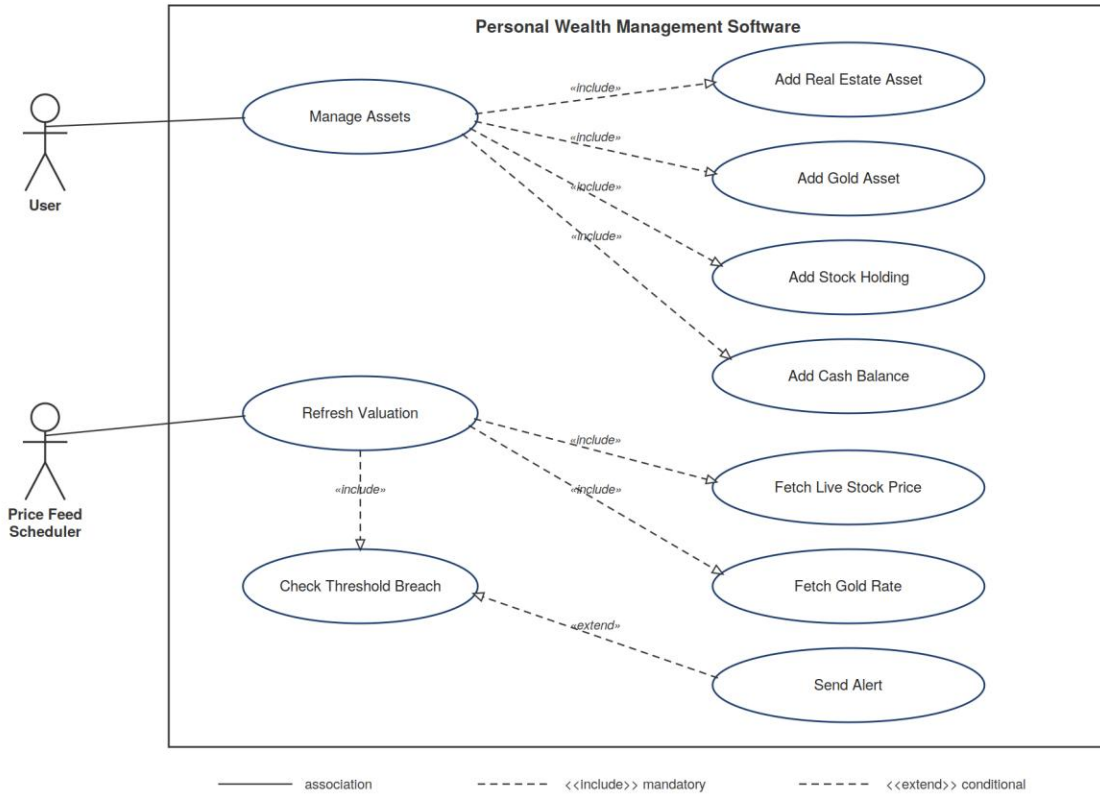
For this to count as done: every High-priority functional requirement is implemented and verified, there are no critical NFR failures, and the RTM shows all planned test cases passing.

The acceptance test suites are: Authentication, Asset Management, Valuation & Price Feed Integration, Dashboard & Reporting, Alerts, Performance, and Security.

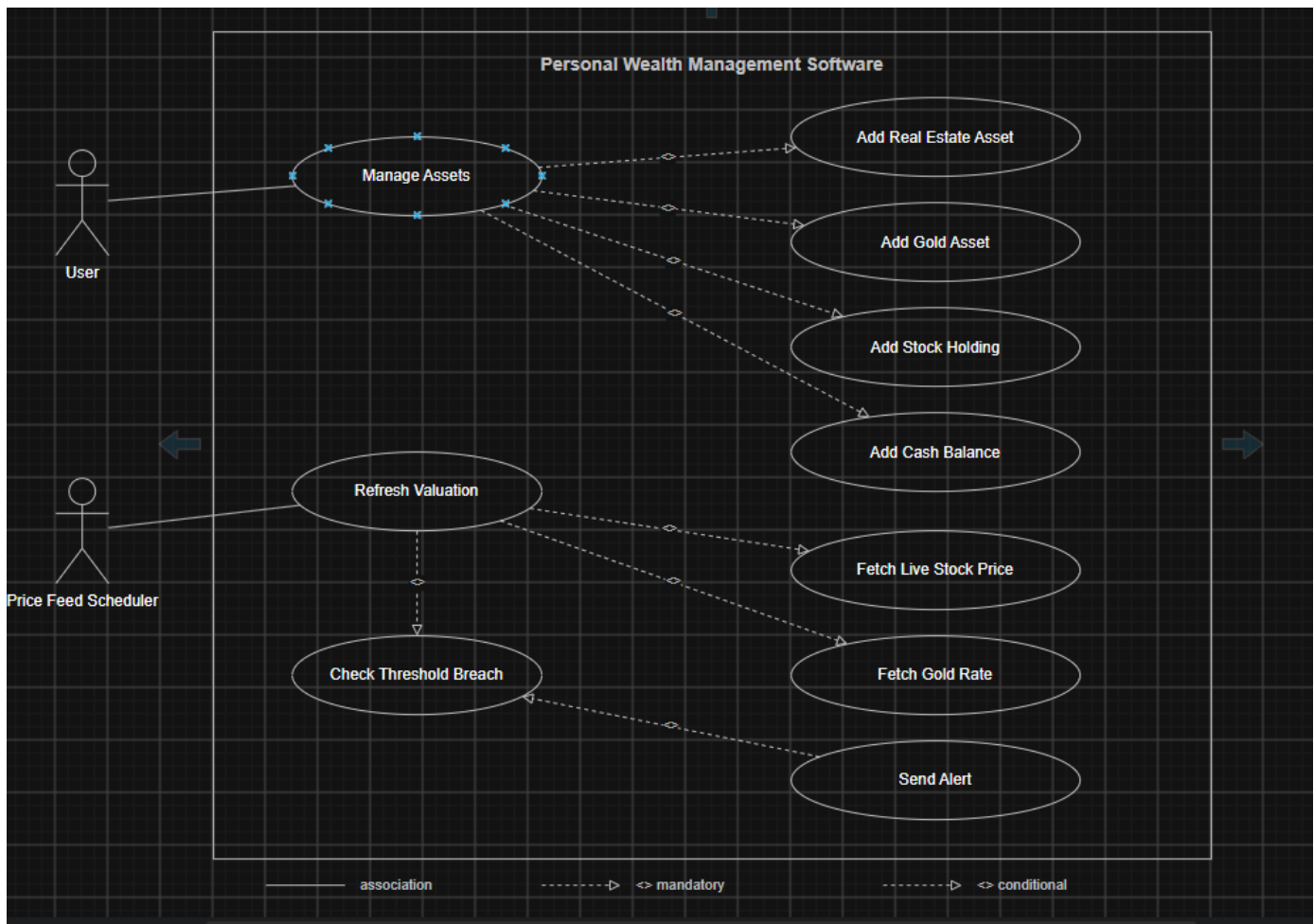
7. System Models and Diagrams

7.1 UML Use-Case Diagram





Use-Case Diagram 2: Manage Assets («include» per asset type) and valuation refresh with threshold-based alerting («extend»).



8.Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
PWM-F-001	User registration	4.1	AuthModule	TC-Auth-01	N	Password policy
PWM-F-002	User login (JWT)	4.1	AuthModule	TC-Auth-02	N	
PWM-F-003	Forgot-password reset	4.1	AuthModule	TC-Auth-03	N	Email service
PWM-F-010	Add real estate asset	4.2	AssetModule	TC-RE-01	N	
PWM-F-011	Add gold asset	4.2	AssetModule	TC-Gold-01	N	
PWM-F-012	Add stock holding	4.2	AssetModule	TC-Stock-01	N	Price-feed lookup
PWM-F-013	Add cash/bank balance	4.2	AssetModule	TC-Cash-01	N	
PWM-F-014	Edit/delete asset	4.2	AssetModule	TC-Asset-01	N	
PWM-F-015	Validate asset inputs	4.2	AssetModule	TC-Asset-02	N	
PWM-F-020	Auto stock valuation refresh	4.3	ValuationService	TC-Val-01	N	
PWM-F-021	Auto gold valuation refresh	4.3	ValuationService	TC-Val-02	N	
PWM-F-022	Manual real-estate valuation update	4.3	ValuationService	TC-Val-03	N	
PWM-F-023	Historical asset valuation	4.3	ValuationService	TC-Val-04	N	
PWM-F-024	Cached price + stale flag	4.3	ValuationService	TC-Val-05	N	
PWM-F-030	Net worth dashboard total	4.4	DashboardUI	TC-Dash-01	N	
PWM-F-031	Asset allocation chart	4.4	DashboardUI	TC-Dash-02	N	

-- continued

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
PWM-F-032	Historical net-worth chart	4.4	DashboardUI	TC-Dash-03	N	
PWM-F-033	CSV/PDF export	4.4	ReportModule	TC-Export-01	N	
PWM-F-040	Alert threshold configuration	4.5	AlertService	TC-Alert-01	N	
PWM-F-041	Threshold-breach notification	4.5	AlertService	TC-Alert-02	N	
PWM-F-042	Alert throttling	4.5	AlertService	TC-Alert-03	N	
PWM-NF-001	Dashboard load time $\leq$ 2s	5	DashboardUI / API	TC-Perf-01	N	
PWM-NF-002	Refresh $\leq$ 5 min for 500 assets	5	ValuationService	TC-Perf-02	N	
PWM-NF-003	99.5% monthly uptime	5	Infra	Ops reports	N	
PWM-NF-004	Responsive 360px to desktop	5	Frontend	TC-UX-01	N	
PWM-NF-005	Support 1,000 registered users	5	Backend / DB	TC-Scale-01	N	
PRJ-SR-001	TLS 1.2+ everywhere	5.1.2	Infra / API	TC-Sec-01	N	
PRJ-SR-002	Salted one-way password hash	5.1.2	AuthModule	TC-Sec-02	N	
PRJ-SR-003	JWT expiry + logout invalidation	5.1.2	AuthModule / API	TC-Sec-03	N	
PRJ-SR-004	Per-user data authorization	5.1.2	AuthModule / API	TC-Sec-04	N	
PRJ-SR-005	Lock after 5 failed logins	5.1.2	AuthModule	TC-Sec-05	N	